Машинное обучение



Козлов Денис Юрьевич, к.ф.-м.н, доцент АлтГУ, Барнаул, 2026

План

1. Орг. информация о курсе
2. Инструментарий (Python, Jupyter Notebook из Anaconda, Google Colab)
3. Литература и источники
4. Некоторые рекомендуемые онлайн-курсы
5. Описательная статистика
6. Кросс-таблица
7. Группировка данных
8. Визуализация данных

Орг. информация о курсе

Орг. информация о курсе

•2 семестра, в 1м и 2 семестрах по 14 часов лекций, 24 часа лабораторных работ, Зачет и Экзамен.

• Курс на образовательном портале университета https://portal.edu.asu.ru/course/view.php?id=11410.

•Кодовое слово «МО».

Инструментарий (Python, Jupyter Notebook из Anaconda, Google Colab)

Инструментарий (Python, Jupyter Notebook из Anaconda, Google Colab)

- Язык программирования Python (версия не ниже 3.9). Загрузка по адресу https://www.python.org/downloads/.

- Anaconda – дистрибутив для языка Python, специализированный для машинного обучения. Загрузка по адресу

https://www.anaconda.com/products/individual. В основном, будет нужен Jupyter Notebook

- Google Colab – свободный онлайн-сервис от Google, где каждый, у кого есть Google-аккаунт, может попробовать силы в машинном обучении, практически повторяет интерфейс Jupyter Notebook.

Литература и источники

Литература и источники

1. Марк Лутц. Изучаем Python, 4-е издание. — Перевод с английского. — СПб.: Символ-Плюс, 2010. — 1280 с.
2. Python 3.x documentation (https://docs.python.org/3/).
3. Жерон, О. Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow: концепции, инструменты и техники для создания интеллектуальных систем. Пер. с англ. - СпБ.: ООО "Альфа-книга': 2018. - 688 с.
4. Дейтел П., Дейтел Х. Python: Искусственный интеллект, большие данные и облачные вычисления. — СПб.: Питер, 2020. — 864 с.

Литература и источники

5. Джоши П. Искусственный интеллект с примерами на Python. : Пер. с англ. - СПб. : ООО "Диалектика", 2019. - 448 с.
6. Силен Д., Мейсман А., Али М. Основы Data Science и Big Data. Python и наука о данных, СПб.: Питер, 2018 г. — 336 с.
7. Ын А., Су К. Теоретический минимум по Big Data. Всё, что нужно знать о больших данных. — СПб.: Питер, 2019. — 208 с.
8. http://www.machinelearning.ru/ — профессиональный информационно-аналитический ресурс, посвященный машинному обучению, распознаванию образов и интеллектуальному анализу данных.

Некоторые рекомендуемые онлайн-курсы на образовательной платформе Stepik

Некоторые рекомендуемые онлайн-курсы на Stepik

1. Введение в Data Science и машинное обучение https://stepik.org/course/4852
2. Машинное обучение https://stepik.org/course/8057/
3. Нейронные сети https://stepik.org/course/401/
4. Нейронные сети и компьютерное зрение https://stepik.org/course/50352/

Основы Python для работы с данными

Данные	Примеры
Номинальные (N)	Квартира, дом, аренда
Порядковые (O)	Ненависть< антипатия < нейтралитет < симпатия < любовь
Численные (I)	100 рублей
Соотношения (R)	5432 > 4321

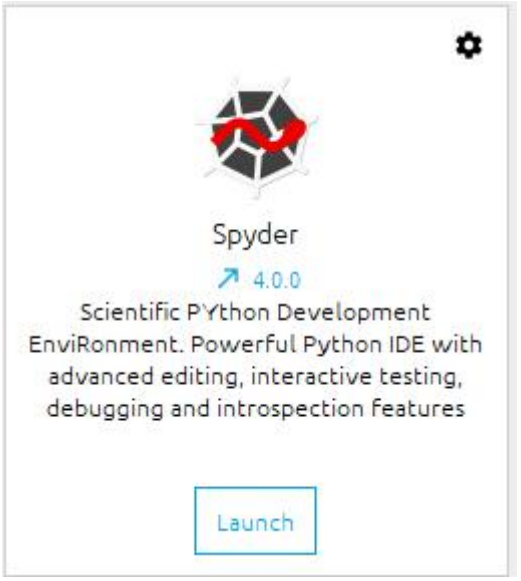
Основы Python для работы с данными

Аналитики чаще всего используют две среды разработки для Python:

1) Jupyter Notebook

2) Spyder





Основы Python для работы с данными

Среда разработки Jupyter Notebook является самой распространённой.

По сути представляет собой интерактивную тетрадку в формате .ipynb содержащую как текстовую информацию, так и блоки кода.

Содержит 3 типа полей данных:

1) Code – исполняемый код

2) Markdown – текстовый блок (с поддержкой гипертекстовой разметки)

3) Heading - заголовки

Пример блока Code

```
In [1]: a = "Russia"
        print(a)
```

Блоки кода запускаются комбинацией Shift + Enter

или нажатием кнопки на панели



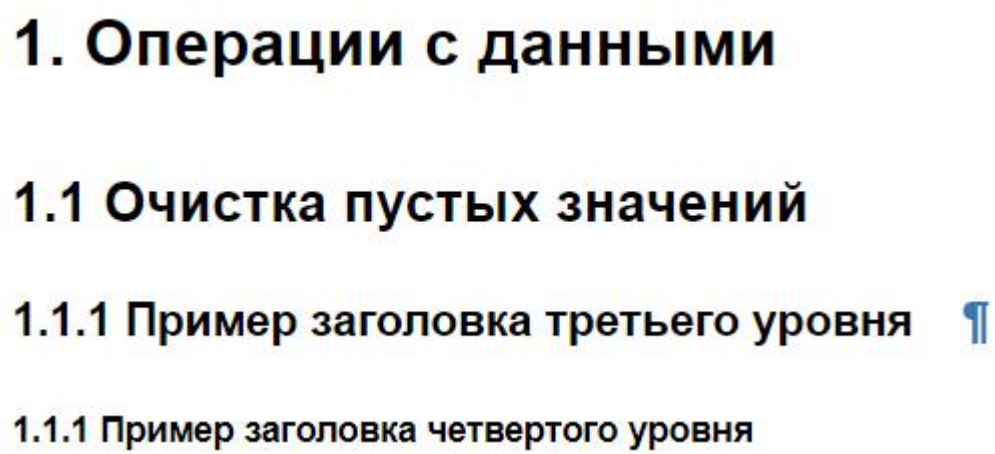
Пример блока Markdown

В этой лабораторной мы рассмотрим некоторые из основных продвинутых операций Pandas.

- **groupby:** группирует коллекции записей, которые имеют одинаковое значение для одного набора полей, а затем вычисляет совокупную статистику по оставшимся полям
- **pivot:** похож на groupby, за исключением того, что результаты представлены немного по-другому ...
- **merge:** *join* данные из пары DataFrame в один DataFrame.

Поддерживает огромное количество тегов для форматирования (списки, выделение, формулы и т.п.)

Пример блока Heading



Уровень заголовка задается числом символов # перед ним

1.1 Очистка пустых значений

1.1.1 Пример заголовка третьего уровня

1.1.1 Пример заголовка четвертого уровня

Фреймворк Anaconda содержит большое число самых необходимых библиотек для работы с данными.

Тем не менее, иногда возникает потребность выполнить установку, удаление или обновление какой-то библиотеки.

Сделать это можно тремя путями:

- Через командную строку Anaconda
- Через Anaconda Navigator, вкладка Environments
- Через вызов команд операционной системы напрямую из Jupyter Notebook

В поиске Windows можно написать Anaconda Prompt и открыть командную строку, которая будет найдена в режиме системного администратора.

После этого просто написать команду

pip install имя_библиотеки

Например

pip install plotly

Установка библиотек из Jupyter Notebook:

В данном случае, вам просто необходимо написать ту же команду на установку в блоке типа Code, предварительно поставив перед ней восклицательный знак:

!pip install имя_библиотеки

```
!pip install plotly
```

Основы Python для работы с данными

Для того, чтобы использовать библиотеки в своей работе, вам необходимо сначала их импортировать в рабочую область.

Для этого, в блоке типа Code необходимо написать

Import имя_библиотеки – например import pandas

```
import pandas
```

Как было сказано выше – каждая команда библиотеки – это по сути функция.

Часто возникает ситуация, что в разных библиотеках, функции носят одинаковые названия и если не указать ИЗ КАКОЙ КОНКРЕТНО библиотеки необходимо использовать функцию – можно получить совершенно непредсказуемое поведение вашего кода.

Для решения этой проблемы, функции можно импортировать как алиас, дописывая который перед командой, вы указываете, из какой библиотеки вы ее используете.

Для этого команду импорта модифицируют следующим образом:

Import имя_библиотеки as название алиаса

```
# импорт библиотек
import numpy as np
import pandas as pd
```

А при использовании – дописывают алиас перед командой

```
df = pd.read_csv('marketing_eng.csv')
```

Anaconda по умолчанию использует в качестве своего рабочего каталога по умолчанию каталог пользователя, под которым вы вошли в систему.

В общем случае, это путь

C:\Users\имя_пользователя

Потому, все ваши рабочие файлы и данные следует копировать в этот каталог, если нет планов сменить установки.

Основы Python для работы с данными

Типы данных	Примеры
Строки (str)	`x = "Сторока"`
Числа (Numeric)	`int = 5` `float = 36.6`
Списки (list)	x = ["яблоки", "бананы", "апельсины"]
Словари (dict)	x = {"name" : "Вася", "age" : 23} Другими словами Name: Вася Age: 23
Булевы (bool)	`x = True`

Основы Python для работы с данными

Python поддерживает множество форматов считывания данных

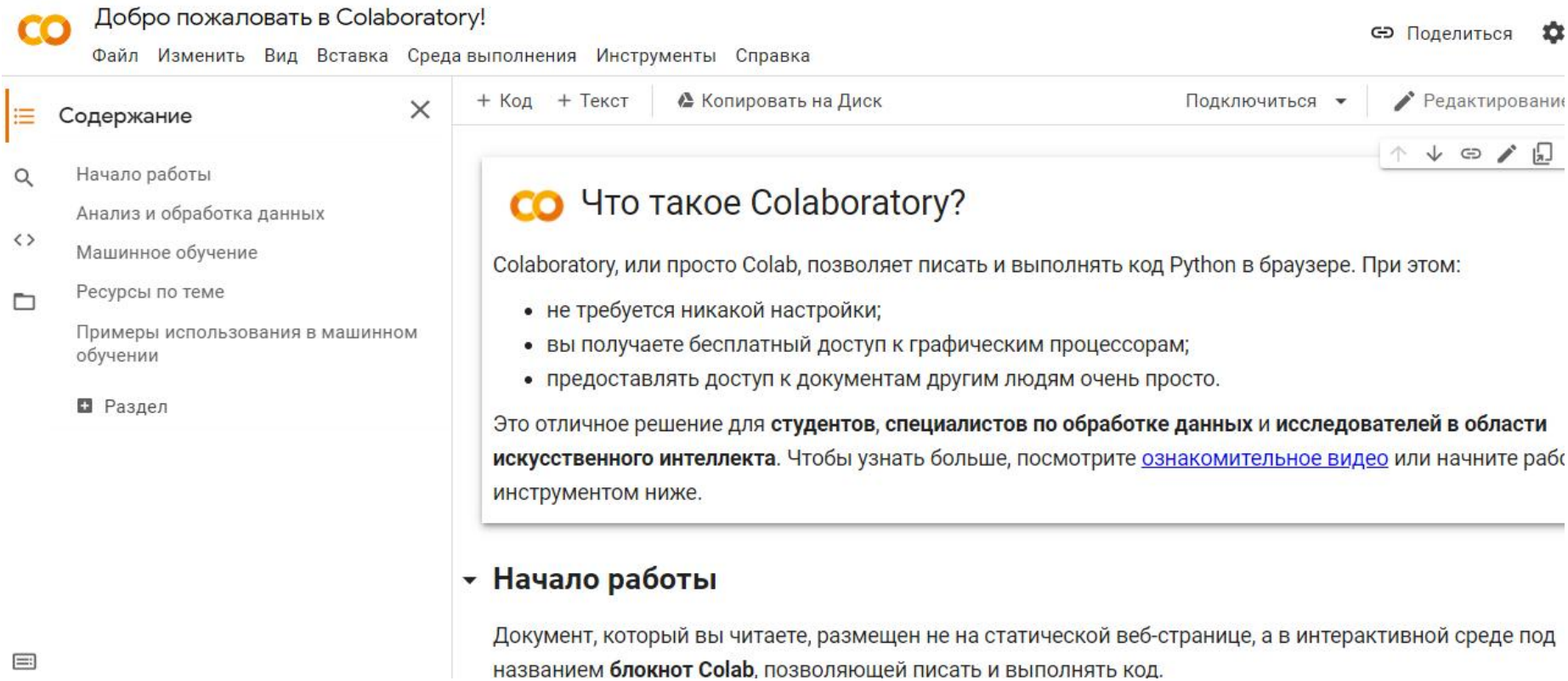
- pandas.read_csv()– это основная функция

Файл в т.ч. может быть и URL

- pandas.read_csv(“http://host/file.csv”, sep=“;”)

Google Colab

Google Colab (https://colab.research.google.com/) - бесплатный облачный сервис для студентов, специалистов по обработке данных и исследователей в области искусственного интеллекта.



Описательная статистика

Метод describe из библиотеки pandas показывает основные статистические характеристики данных по каждому числовому признаку (типы int64 и float64):

3) 1) число непропущенных значений, 2) среднее,

4) стандартное отклонение,

5) диапазон,

6) 1, 2 (медиану), 3 квартили.

```
df.describe()
```

Описательная статистика

Чтобы посмотреть статистику по нечисловым признакам, нужно явно указать интересующие нас типы в параметре include. Можно также задать include='all', чтоб вывести статистику по всем имеющимся признакам:

```
df.describe(include=['object', 'bool'])
```

или

```
df.describe(include='all')
```

Описательная статистика

Пример:

```
df.describe(include=['object', 'bool'])
```

	EDUCATION	MARITAL_STATUS	ORG_TP_STATE	JOB_DIR
count	15223	15223	13856	13856
unique	7	5	5	10
top	Среднее специальное	Состою в браке	Частная компания	Участие в основ. деятельности
freq	6518	9416	6523	11452

В строке count указано количество непустых значений переменных.

unique - количество уникальных уровней, которые принимает переменная

top - самый частый уровень встречается freq раз.

Описательная статистика

Для категориальных (тип object) и булевых (тип bool) признаков можно воспользоваться методом value_counts.

Пример:

```
df['EDUCATION'].value_counts()
```

```
Среднее специальное                6518
Среднее                            4679
Высшее                             3133
Неоконченное высшее                 532
Неполное среднее                    340
Два и более высших образования       20
Ученая степень                        1
Name: EDUCATION, dtype: int64
```

Так можно увидеть распределение целевой переменной EDUCATION.

Описательная статистика

Укажем значение параметра normalize=True, чтобы посмотреть не абсолютные частоты, а относительные. Например, доли лиц с разным семейным статусом.

Пример:

```
df['MARITAL_STATUS'].value_counts(normalize=True)
```

```
Состою в браке          0.618538
Не состоял в браке      0.237929
Разведен(а)             0.081784
Вдовец/Вдова            0.038626
Гражданский брак        0.023123
Name: MARITAL_STATUS, dtype: float64
```

Описательная статистика

По отдельности самые часто используемые методы, сгруппированные в describe:

- count
- mean
- median

Описательная статистика

Отбор данных по условию:

```
df[df['OWN_AUTO'] == 1].mean()
```

```
TARGET                0.137931
AGE                  38.585076
SOCSTATUS_WORK_FL     0.967213
SOCSTATUS_PENS_FL     0.059921
GENDER                0.304127
PERSONAL_INCOME    18301.039005
OWN_AUTO               1.000000
WORK_TIME             80.890251
LOAN_MAX_DLQ_AMT     284.343929
dtype: float64
```

каковы средние характеристики владельцев автомобилей?

Описательная статистика

Отбор данных по условию:

```
df[df['OWN_AUTO'] == 1]['PERSONAL_INCOME'].mean()
```

```
18301.03900508762
```

Каков средний доход владельцев автомобилей?

Описательная статистика

Отбор данных по условию:

```
df[df['AGE']> 30]['AGE'].count()
```

```
11405
```

Сколько клиентов банка старше 30 лет?

Кросс-таблицы (таблицы сопряженности)

Допустим, мы хотим посмотреть, как наблюдения в нашей выборке распределены в контексте двух признаков — GENDER и MARITAL_STATUS. Для этого мы можем построить таблицу сопряженности, воспользовавшись методом crosstab:

```
pd.crosstab(df['EDUCATION'], df['GENDER'])
```

GENDER	0	1
EDUCATION		
Высшее	978	2155
Два и более высших образования	10	10
Неоконченное высшее	186	346
Неполное среднее	133	207
Среднее	1710	2969
Среднее специальное	2242	4276
Ученая степень	0	1

Кросс-таблицы (таблицы сопряженности)

Допустим, мы хотим посмотреть, как наблюдения в нашей выборке распределены в контексте двух признаков — GENDER и MARITAL_STATUS. Для этого мы можем построить таблицу сопряженности, воспользовавшись методом crosstab:

```
pd.crosstab(df['EDUCATION'], df['GENDER'], normalize=True)
```

GENDER	0	1
EDUCATION		
Высшее	0.064245	0.141562
Два и более высших образования	0.000657	0.000657
Неоконченное высшее	0.012218	0.022729
Неполное среднее	0.008737	0.013598
Среднее	0.112330	0.195034
Среднее специальное	0.147277	0.280891
Ученая степень	0.000000	0.000066

Кросс-таблицы (таблицы сопряженности)

Допустим, мы хотим посмотреть, каков для двух признаков — GENDER и MARITAL_STATUS средний возраст заемщиков:

```
pd.crosstab(df['EDUCATION'], df['GENDER'], values=df['AGE'], aggfunc=np.mean)
```

GENDER	0	1
EDUCATION		
Высшее	37.518405	38.628306
Два и более высших образования	40.600000	41.900000
Неоконченное высшее	29.973118	29.936416
Неполное среднее	39.571429	50.289855
Среднее	39.630409	43.884473
Среднее специальное	37.836307	42.046305
Ученая степень	NaN	54.000000

Сводные таблицы

В Pandas за сводные таблицы отвечает метод pivot_table, который принимает в качестве параметров:

- values – список переменных, по которым требуется рассчитать нужные статистики,
- index – список переменных, по которым нужно сгруппировать данные,
- aggfunc — то, что нам, собственно, нужно посчитать по группам — сумму, среднее, максимум, минимум или что-то ещё.

Сводные таблицы

```
df.pivot_table(['AGE','WORK_TIME','PERSONAL_INCOME'], ['OWN_AUTO', 'GENDER'], aggfunc='mean')
```

		AGE	PERSONAL_INCOME	WORK_TIME
OWN_AUTO	GENDER			
0	0	38.116216	16110.229739	72.451647
	1	41.728093	12055.115339	435.543015
1	0	38.194963	19095.190089	76.089925
	1	39.477695	16483.938662	92.150391
2	0	22.000000	15000.000000	36.000000

Группировка данных

В общем случае группировка данных в Pandas выглядит следующим образом:

```
df.groupby(by=grouping_columns)[columns_to_show].function()
```

1) К датафрейму применяется метод groupby, который разделяет данные по grouping_columns – признаку или набору признаков.

2) Индексируем по нужным нам столбцам (columns_to_show).

3)К полученным группам применяется функция или несколько функций.

Группировка данных

```
df.groupby(by=['TARGET'])['AGE', 'WORK_TIME', 'PERSONAL_INCOME'].agg([np.std, np.max])
```

	AGE		WORK_TIME		PERSONAL_INCOME	
	std	amax	std	amax	std	amax
TARGET						
0	11.697037	67	26079.479331	2867959.0	8794.221229	250000.0
1	10.290703	65	75.522526	864.0	10217.147092	150000.0

Визуализация данных

matplotlib - это библиотека двумерной графики для языка программирования python с помощью которой можно создавать высококачественные рисунки различных форматов.

Мы будем использовать конкретный модуль из этой библиотеки, который отвечает за плоские графики – pyplot.

Распространенный способ его подключения таков:

```
import matplotlib.pyplot as plt
```

Это даст возможность использовать короткий псевдоним plt при обращении к модулю для задействования различных графических возможностей.

Визуализация данных

Seaborn — это по сути более высокоуровневое API на базе библиотеки matplotlib. Seaborn содержит более адекватные дефолтные настройки оформления графиков. Также в библиотеке есть достаточно сложные типы визуализации, которые в matplotlib потребовали бы большого количества кода.

```
import seaborn as sns
```

Визуализация данных

Первое, что обычно рекомендуется делать в плане визуализации данных, - построить диаграмму рассеяния - скаттерограмму для выбранных переменных (Иногда это занимает некоторое время, особенно при больших объемах данных).

```
import matplotlib.pyplot as plt

import seaborn as sns

sns.pairplot(df, kind="scatter")
```

Визуализация данных

Полезный тип графиков boxplot. Boxplot состоит из коробки, усиков и точек. Коробка показывает внутриквартильный размах распределения, то есть соответственно 25 % (Q1) и 75 % (Q3). Черта внутри коробки обозначает медиану распределения.

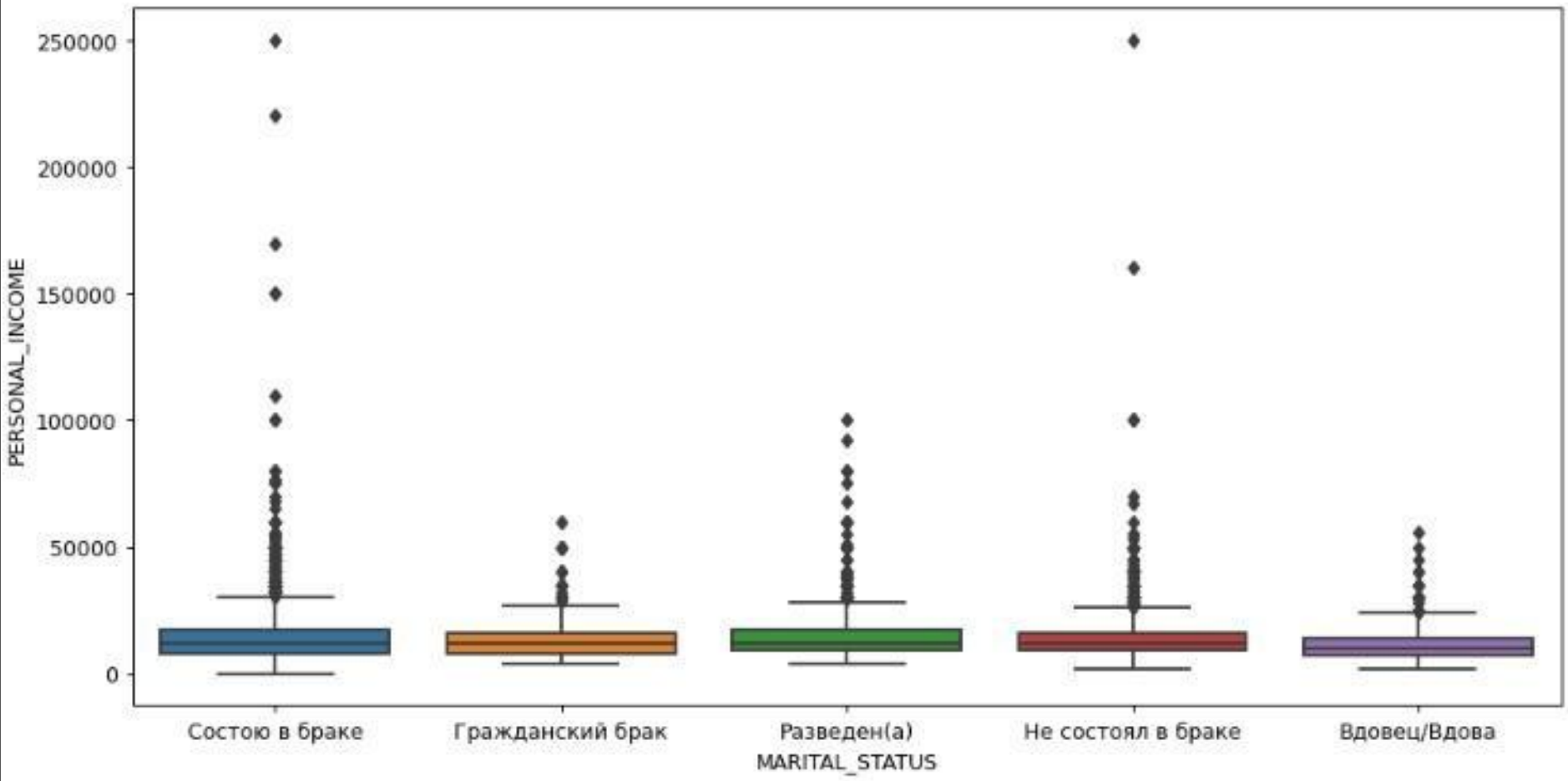
Усы отображают весь разброс точек кроме выбросов, то есть минимальные и максимальные значения, которые попадают в промежуток (Q1 - 1.5*IQR, Q3 + 1.5*IQR), где IQR = Q3 - Q1 — внутриквартильный размах.

Точками на графике обозначаются выбросы (outliers) — те значения, которые не вписываются в промежуток значений, заданный усами графика.

Визуализация данных

```
plt.figure(figsize=(12,6), dpi=90)

sns.boxplot(x = df['MARITAL_STATUS'], y = df['PERSONAL_INCOME'])
```



Визуализация данных

Иногда наглядней изобразить распределение с помощью countplot.

```
plt.figure(figsize=(12,6), dpi=90)

sns.countplot(data = df, x = 'MARITAL_STATUS', hue = 'GENDER')
```

Визуализация данных

